

SoK: Más allá del catálogo de amenazas

Una taxonomía empírica de fallas de gobernanza de autorización en servers MCP

Matías Zabaljáuregui — trabajo de doctorado en curso

Sometido (double-blind) — SaTML 2027

Curso de Seguridad — sesión invitada

¿Por qué importa esto ahora?

- **Model Context Protocol (MCP):** estándar de Anthropic para que un agente LLM invoque *tools* externas (bases de datos, APIs, filesystems, wallets, historias clínicas)
- Cada *tool* que un server MCP expone es, en los hechos, **una nueva superficie de autorización**
- El agente decide *qué tool llamar* en base a lenguaje natural — a veces influenciado por contenido que no controla el usuario
- Pregunta de fondo: **¿quién decidió qué puede hacer este agente, y cómo se hizo cumplir esa decisión?**

El estado de la literatura (y su hueco)

Dos géneros dominan la seguridad de MCP hasta ahora:

1. **Surveys de amenazas** (taxonomías de ataques) — no miran código real
2. **Scans automatizados a gran escala** — miden prevalencia de bugs conocidos, no *diseño* de gobernanza

Ninguno responde lo que necesita un practicante: cuando un server *sí* intenta gobernar lo que un agente puede hacer, ¿cómo se ve eso cuando funciona? ¿Y cómo falla, específicamente?

Este trabajo

Auditoría manual de **89 repositorios MCP públicos y distintos**
(91 pasadas de auditoría, 15 rondas estratificadas por dominio)

Cada hallazgo clasificado por **tier de evidencia**:

- Tier 1 — lectura de código
- Tier 2 — ejecutado con el *sink* final mockeado
- Tier 3 — ejecutado de punta a punta contra una reconstrucción aislada

No es un scanner que tira un número. Es lectura de código, ronda por ronda, dominio por dominio (salud, fintech, CRM, infraestructura, identidad...).

Hallazgo central

Menos del 10% de los repositorios tiene un bug de código explotable.
La falla dominante no es un error de programación — **es una decisión de autorización que nunca se tomó.**

Servers que mapean 1:1 toda la API subyacente (lectura y control) sobre tools de MCP, sin reducir superficie por mínimo privilegio, con una única credencial compartida.

No falta código. Falta una decisión de diseño.

Taxonomía de tres capas + un eje ortogonal

Capa	Pregunta que responde	Falla característica
1. Exposición de control-plane	¿Qué parte de la API subyacente se expone como tool?	Mapeo 1:1 sin reducción por privilegio mínimo
2. Ejecución y confianza	¿Con qué identidad y qué confirmación se ejecuta?	Credencial compartida, sin gate humano en mutaciones críticas
3. Señalización de riesgo	¿El server le dice al cliente MCP qué tan riesgosa es una tool?	Ausencia de <code>destructiveHint</code> / <code>readOnlyHint</code>
Ortogonal: integridad de implementación	Cuando <i>sí</i> se intenta un límite, ¿está bien construido?	El mecanismo existe pero no cumple lo que promete

El marco teórico ya existe — y calza

- **Confused Deputy** (Hardy, 1988): un programa que ejecuta con más autoridad que la que le confirió quien lo invocó
→ aquí, el agente LLM es un *deputy* sub-especificado entre la intención del usuario y la autoridad total de la tool
- **Indirect Prompt Injection** (Greshake et al., 2023): contenido que el agente ingiere como dato puede redirigir qué hace con su autoridad
- **ABAC / separación PDP-PEP** (Policy Decision Point / Policy Enforcement Point, ej. Open Policy Agent + Rego): el vocabulario arquitectónico estándar para *externalizar* la decisión de política

No inventamos estos conceptos — el corpus real es donde se vuelven concretos.

Caso 1 — Cuando la contención declarada no es real

Server MCP de historias clínicas: expone una tool que evalúa JavaScript arbitrario del caller contra el registro FHIR completo del paciente, usando el módulo `vm` de Node.js como "sandbox"

- La propia documentación de Node.js dice: *"the vm module is not a security mechanism"*
- Reconstruimos el escape (misma técnica que CVEs conocidos contra `vm2`) contra un objeto sintético — **no** contra el server en producción
- Resultado: acceso a `process` del host, en microsegundos, sin evadir timeout
- **Sin ninguna anotación de riesgo** frente a las otras 4 tools del mismo server

Esto no es "no se intentó contener". Es: **se declaró una intención de sandboxing, y el primitivo elegido no la cumple.**

Caso 2 — La tool que existe para saltar la autorización

Server MCP de Salesforce: 14 tools, 7 correctamente anotadas `readOnlyHint: true`, enrutadas por la REST API que respeta field-level security

Una octava tool, `salesforce_execute_anonymous`, ejecuta Apex arbitrario en **modo sistema** — sin las reglas de compartición que sí aplican en las tools de lectura del mismo server

Su propia descripción invita a usarla: *"this tool can be used [...] if the operation is achievable using Apex code"*

Sin anotación, sin gate. **No es una tool de escritura sin freno — es una tool diseñada para saltar el modelo de autorización de la propia plataforma.**

Cuando la gobernanza sí está bien hecha

No es un patrón único — es un catálogo de **10 patrones**, cuatro de ellos recurrentes de forma **independiente** en proyectos que nunca se vieron entre sí:

1. Delegación dinámica de identidad (token del caller, no credencial compartida)
 2. Credencial que nunca cruza el protocolo (solo un alias opaco)
 3. **Decisión de política externalizada y fail-closed** ← el que nos interesa hoy
 4. Confirmación auto-derivada de la anotación de riesgo (no opt-in)
 5. Límites de blast-radius persistentes e independientes del caller
- ...

Ningún repositorio combina más de 3–4 de los 10.

El patrón 3, en código real

`lightning-wallet-mcp` : antes de ejecutar un pago, un webhook de pre-ejecución consulta un endpoint de política externo

- Si el endpoint no responde o la respuesta es irreconocible → **deny por default**
- La decisión de "¿puede este caller hacer este pago?" vive **afuera** del código que ejecuta el pago

Esto es, literalmente, una **separación PDP/PEP artesanal** — construida sin que el autor necesariamente conociera el término, para resolver el mismo problema que Open Policy Agent formaliza.

No es una idea de laboratorio. Ya apareció, de forma convergente, en producción.

¿Por qué OPA es un buen "caso" para MCP?

Cuatro razones, ancladas en lo que el corpus mostró — no en preferencia estética:

1. **El hallazgo central es un problema de política, no de código.** "Nadie tomó la decisión" es exactamente el vacío que un PDP externalizado está diseñado para llenar: la política se declara aparte de la lógica de negocio, en vez de vivir implícita (o ausente) en cada handler de tool.
2. **MCP ya expone el punto de decisión correcto, gratis.** Cada tool call es un mensaje JSON-RPC estructurado — nombre de tool, argumentos, identidad del caller — interceptable en un único punto antes de la ejecución. Es el *enforcement point* que un PEP necesita, sin inventar nada.

¿Por qué OPA es un buen "caso" para MCP? (cont.)

3. El protocolo ya trae metadata de riesgo lista para ser input de una policy.

`readOnlyHint`, `destructiveHint`, `idempotentHint`, `openWorldHint` son exactamente el tipo de atributo que un motor ABAC como Rego evalúa — y Layer 3 de nuestra taxonomía muestra que casi nadie los usa sistemáticamente. Rego puede *forzar* su uso aunque el desarrollador del server lo haya omitido.

4. Es retrofit-able sin tocar el server. Porque MCP es un protocolo de red (no una librería embebida), un PEP puede vivir como *proxy/sidecar* delante de cualquiera de los 89 servers del corpus — incluidos los que fallaron Layer 1/2 — sin reescribir una línea de su código.

Honestidad metodológica: qué NO hicimos

- **No construimos ningún prototipo con OPA.** No hay proxy, no hay integración con Rego, no hay benchmark de latencia.
- OPA aparece en este trabajo como **vocabulario arquitectónico** (Related Work) y como el patrón que **ya observamos, informalmente, en código real** (`lightning-wallet-mcp`) — no como un sistema evaluado.
- Construir y evaluar ese sistema es **un paper distinto** (de sistemas, no un SoK) — meses de ingeniería real que no salen de otra ronda de auditoría.

Distinguir "tengo la fundamentación teórica" de "tengo la arquitectura resuelta" es, en sí, una disciplina de este trabajo.

La severidad tampoco es solo código

Un policy engine correcto no alcanza si la política no conoce la semántica de la plataforma:

- Un `delete` en un server de contabilidad puede ser un *soft-delete* reversible; un `DeleteObject` de S3, no
- Un server de trading puede correr contra un entorno de *paper trading* simulado, no contra dinero real
- Un server puede ejecutar el fetch desde **su propio proceso**, o proxearlo a infraestructura del vendor — cambia contra qué red apunta un SSRF exitoso

Conclusión práctica: la política (tipo OPA) necesita como input esta semántica de plataforma, no solo el nombre de la tool.

Posicionamiento frente a la literatura reciente

Dimensión	SoK previo	DSN'26 (67k servers)	1.899-server study	MCP-DPT	Este trabajo
Auditoría empírica original	X	✓	✓	X	✓
Gobernanza de tool-annotations	X	X	X	X	✓
Catálogo de mitigación positiva	X	X	X	X	✓
Exploit de PoC ejecutado	X	parcial	X	X	✓
Estadísticas de prevalencia a escala	X	✓	✓	X	X

Escala dos órdenes de magnitud menor — profundidad por repositorio, correspondientemente mayor.

Límites de este trabajo

- **Muestra de conveniencia**, no crawl sistemático del registro — sin representatividad estadística garantizada
- **Un solo revisor** — sin segundo codificador, sin inter-rater reliability reportada
- **Disclosure incompleto**: de 5 hallazgos críticos, 2 reportados formalmente, 3 sin canal de disclosure funcional al momento de escribir
- Estas limitaciones se declaran explícitamente en el paper — no se maquillan como fortalezas

Conclusión y hacia dónde va esto

La falla dominante es **arquitectónica, no sintáctica**: ausencia de decisión, no error de programación.

Este corpus alimenta una línea más amplia: un framework de **doblo loop** para hardening continuo de autorización en sistemas agénticos —

- **Red loop**: ¿existe el límite declarado, y está bien construido? (el caso `vm` muestra que "existe" no alcanza)
- **Blue loop**: ¿el mecanismo está activo por default, y resiste la construcción exacta que un atacante probaría?

Un PDP externalizado tipo OPA/Rego es el bloque natural del blue loop — no como conclusión ya probada, sino como próximo paso de investigación con evidencia empírica que lo justifica.

Preguntas

Matías Zabaljáuregui

zabaljauregui@gmail.com

Corpus, scanner y PoCs: liberados al artifact deadline (ver sección *Open Science* del paper)